Entropy: From Information Rate to Loss Function, Explored

Snigdha D. Nagpure

July 9, 2026

Abstract

This essay explores entropy as a concept that connects information theory and machine learning, highlighting its role as a measure of information, uncertainty, and loss. The aim is to present the topic in a way accessible to undergraduate students and readers encountering entropy across different contexts.

Entropy has a rich history and a wide range of applications. In information theory, it describes the average information content, or uncertainty, associated with a probability distribution. In statistics and machine learning, related concepts such as cross-entropy and population cross-entropy appear as loss functions used to train probabilistic models. These ideas arise in different settings and with slightly different interpretations, so the depth and breadth of the concept can escape a learner - especially if the first introduction is through an equation in machine learning. This essay brings these threads together, moving from entropy as an information-theoretic quantity to population cross-entropy as a practical loss function.

1 Journey of Entropy

Outside of physics, entropy was first defined by C. Shannon in his work on information theory to represent the average information rate of a code (a set of symbols and how frequently each is used), measured in bits. He defined it as

$$Entropy = \sum_{symbols} p\, log_2(\frac{1}{p}) \tag{1}$$

where p is the probability of a symbol in the code and $log_2(1/p)$ is the information content of the symbol. The rarer the symbol (that is, the lower the probability of it occurring), the higher its information content.

In probability theory, the concept of entropy is used as a property associated with the probability distribution of a random variable, just like mean and standard deviation are. It quantifies the average uncertainty in the outcome of the random variable. If X is a discrete random variable (r.v.) with probability mass function (PMF) $P(X)$ such that

$$p(x) = P(X = x) \qquad \forall x \tag{2}$$

then entropy of the r.v. X is given by

$$H(X) = -\sum_{x} p(x)\, log(p(x)) \tag{3}$$

$$= -\mathbb{E}_{X\sim P}[log(p(X))] \tag{4}$$

now computed using natural logarithm and measured in nats.

For example, the entropy of a fair six-sided die

$$= -6 * \frac{1}{6} * log(\frac{1}{6}) = log(6)\, nats \tag{5}$$

and the entropy of a fair coin

$$= -2 * \frac{1}{2} * log(\frac{1}{2}) = log(2)\, nats \tag{6}$$

Note that in the examples above, we did not have to throw the die or flip the coin to compute entropy. We simply used the known probability distributions for each.

We could compare random variables by entropy. For example, consider two dice with probability distributions

$$X \sim (\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}) \tag{7}$$

$$Y \sim (\frac{1}{8}, \frac{1}{8}, \frac{3}{8}, \frac{1}{8}, \frac{1}{8}, \frac{1}{8}) \tag{8}$$

where X is for a fair die and Y is for an unfair die. Since some of the outcomes of Y are more likely than others, there is less uncertainty i.e. less entropy compared to X.

A few additional remarks:

a) We could say that entropy of a fair die = $log(6)$ is greater than the entropy of a fair coin = $log(2)$ because there is more uncertainty with a die before the outcome is known. However, comparing r.v.s on the basis of their entropy makes more sense if they each have the same number of outcomes.

b) It is possible to have two r.v.s with different distributions but the same entropy, indicating equal amounts of uncertainty.

c) $\lim_{p\to 0^+}(-p\, log(p)) = 0$ so although $log(0)$ is undefined, the entropy equation works even if $p(x) = 0$ for one or more outcomes.

1.1 Cross-entropy

The idea of **entropy** can now be extended into **cross-entropy**. Here we have two probability distributions, P and Q for the same r.v. X. The cross-entropy of Q relative to P is then given by

$$H(P,Q) = -\mathbb{E}_{X\sim P}[log(q(X))] \tag{9}$$

$$= -\sum_x p(x)\, log(q(x)) \tag{10}$$

Here are a few examples for clarity:

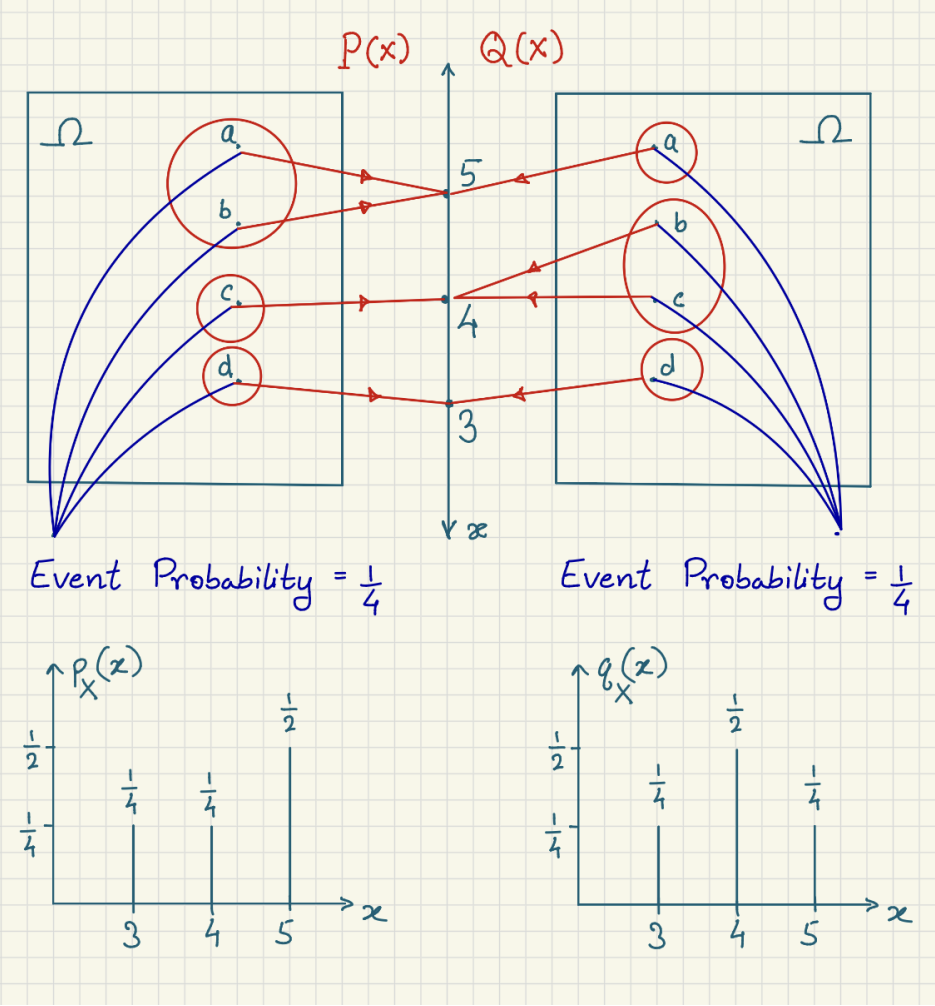

Figure 1: Cross-entropy case 1, different distributions P and Q.

For case 1 in 1, the cross-entropy is

$$H(P,Q) = -(\frac{1}{4}\, log(\frac{1}{4}) + \frac{1}{4}\, log(\frac{1}{2}) + \frac{1}{2}\, log(\frac{1}{4})) \tag{11}$$

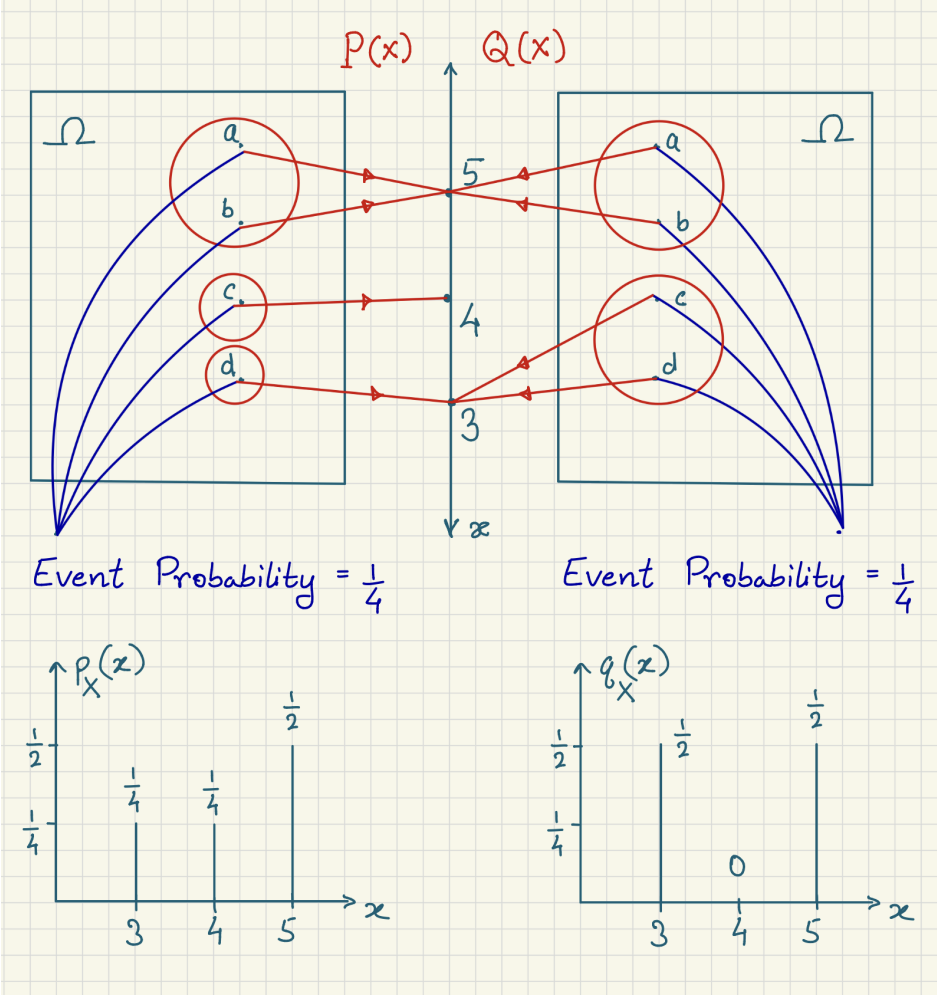

Figure 2: Cross-entropy case 2, Q has different set of outcomes.

For case 2 in 2, the cross-entropy is

$$H(P,Q) = -(\frac{1}{4}\,log(\frac{1}{2}) + \frac{1}{4}\,log(0) + \frac{1}{2}\,log(\frac{1}{2})) \tag{12}$$

Note that

$$\lim_{p\to 0^+}(log\,p) = -\infty \tag{13}$$

so the cross-entropy $\to \infty$.

For case 3 in 3, the cross-entropy is

$$H(P,Q) = -(\frac{1}{4}\,log(\frac{1}{4}) + \frac{1}{4}\,log(\frac{1}{4}) + \frac{1}{2}\,log(\frac{1}{2})) \tag{14}$$

So cross-entropy measures how well the distribution Q aligns with distribution P. If Q assigns low probability to outcomes that are likely under P, cross-entropy will increase. Note that

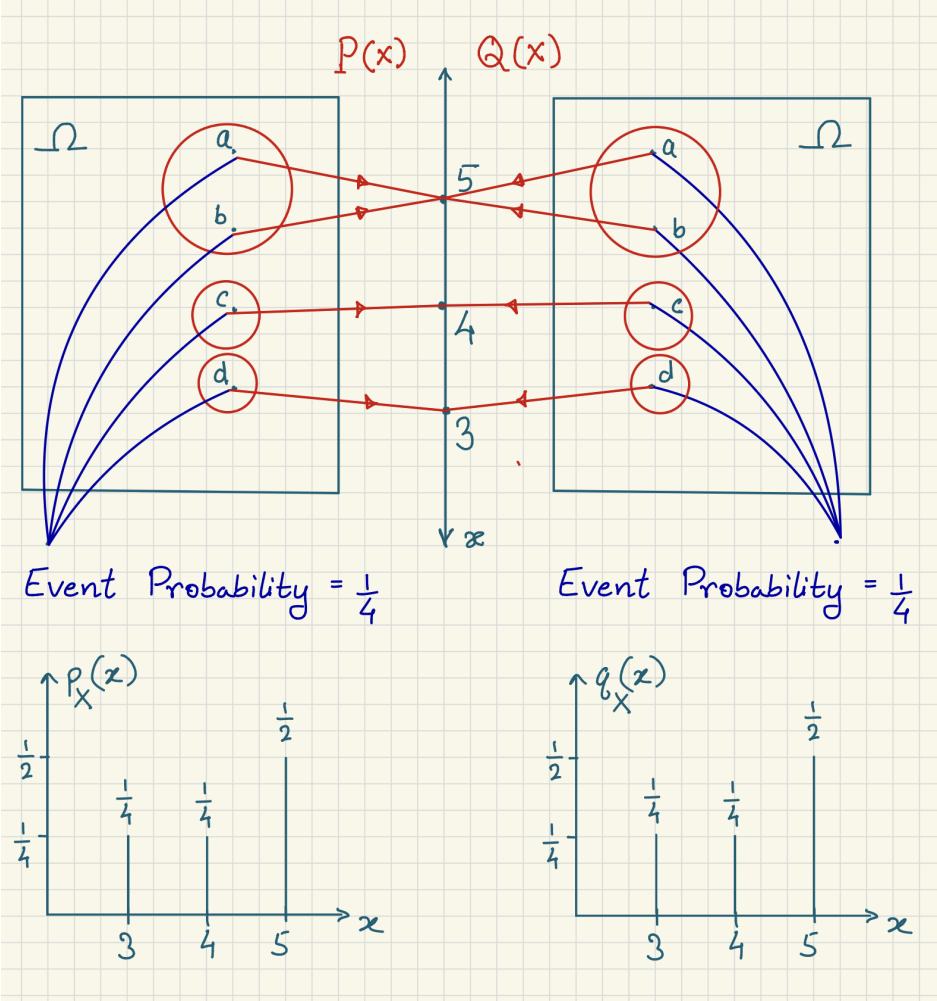

Figure 3: Cross-entropy case 3, P and Q identical.

we cannot however say that cross-entropy measures deviation of Q from P; for that, we have something called KL divergence (mentioned here only for the sake of completeness).

Also note that in all the three examples above, the probability distributions P and Q are known a priori.

In statistics, cross-entropy can be used to measure how well a model's predicted probability distribution matches the true distribution. Let's say we have a model Q to predict the class performance on an exam, with values:

$$q(pass) = 0.75 \tag{15}$$

$$q(fail) = 0.25 \tag{16}$$

Now suppose we have the true class performance for two exams, Biology and Calculus, with values:

Biology

$$p_1(pass) = 0.8 \tag{17}$$

$$p_1(fail) = 0.2 \tag{18}$$

Calculus

$$p_2(pass) = 0.6 \tag{19}$$

$$p_2(fail) = 0.4 \tag{20}$$

Using cross-entropy to quantify the model performance for Biology (model with respect to Biology):

$$H(P_1, Q) = -\mathbb{E}_{X \sim P_1}[log(q(X))] \tag{21}$$

$$= -\sum_i p_1(x_i)\, log(q(x_i)) \tag{22}$$

$$= -[p_1(pass)\, log(q(pass)) + p_1(fail)\, log(q(fail))] \tag{23}$$

$$= -[0.8\, log(0.75) + 0.2\, log(0.25)] \tag{24}$$

$$\approx 0.507\, nats \tag{25}$$

Model performance for Calculus:

$$H(P_2, Q) = -\mathbb{E}_{X \sim P_2}[log(q(X))] \tag{26}$$

$$= -\sum_i p_2(x_i)\, log(q(x_i)) \tag{27}$$

$$= -[p_2(pass)\, log(q(pass)) + p_2(fail)\, log(q(fail))] \tag{28}$$

$$= -[0.6\, log(0.75) + 0.4\, log(0.25)] \tag{29}$$

$$\approx 0.727\, nats \tag{30}$$

So the model aligns more closely with the true Biology distribution than the true Calculus distribution.

Also, the average cross-entropy is

$$-\frac{1}{2}[\mathbb{E}_{X \sim P_1}[log(q(X))] + \mathbb{E}_{X \sim P_2}[log(q(X))]] \approx 0.617\, nats \tag{31}$$

1.2 Population cross-entropy

In machine learning and optimization, cross-entropy is defined as expectation over the **population** and is called **population cross-entropy**

$$= -\mathbb{E}_{X\sim P}[log(q(X))]] \tag{32}$$

If P and Q are known, it can be calculated before any value of X is observed - which is never the case in machine learning. To understand this, consider the scenario in supervised learning. Here, each example is usually a pair (x, y):

x: input such as an image, a sentence

y: label such as "cat", "positive sentiment"

There is a true underlying population distribution given by

$$P_{data}(x, y) \tag{33}$$

and the population cross-entropy gives us a measure to quantify the accuracy of model assigned probabilities. It is like asking if the real world kept producing examples forever, how well would this model assign probabilities to the correct labels on average?

Computed as a loss given by

$$L(\theta) = \mathbb{E}_{(x,y)\sim P_{data}}[-log(q_\theta(y \mid x)] \tag{34}$$

where $q_\theta(y \mid x)$ is the model's predicted probability for label y given x, and θ represents model parameters.

We usually do not have this distribution P_{data}, so we only observe a finite number of examples (like we would observe arriving buses at a bus stop in an effort to compute inter-arrival time):

$$(x^1, y^1), (x^2, y^2), ..., (x^n, y^n) \tag{35}$$

The loss computed using this sample, called **empirical cross-entropy**, is

$$\hat{L}(\theta) = \frac{1}{n}\sum_{j=1}^{n} -log(q_\theta(y^j|x^j)) \tag{36}$$

assuming uniform distribution.

Training the model then means choosing a starting model distribution Q_θ and updating over many samples to a Q that (hopefully) best approximates P. We do this by a process called "learning" in which we minimize $\hat{L}(\theta)$ by updating θ using gradient descent. Note that gradient descent does not guarantee that we arrive at any such θ.

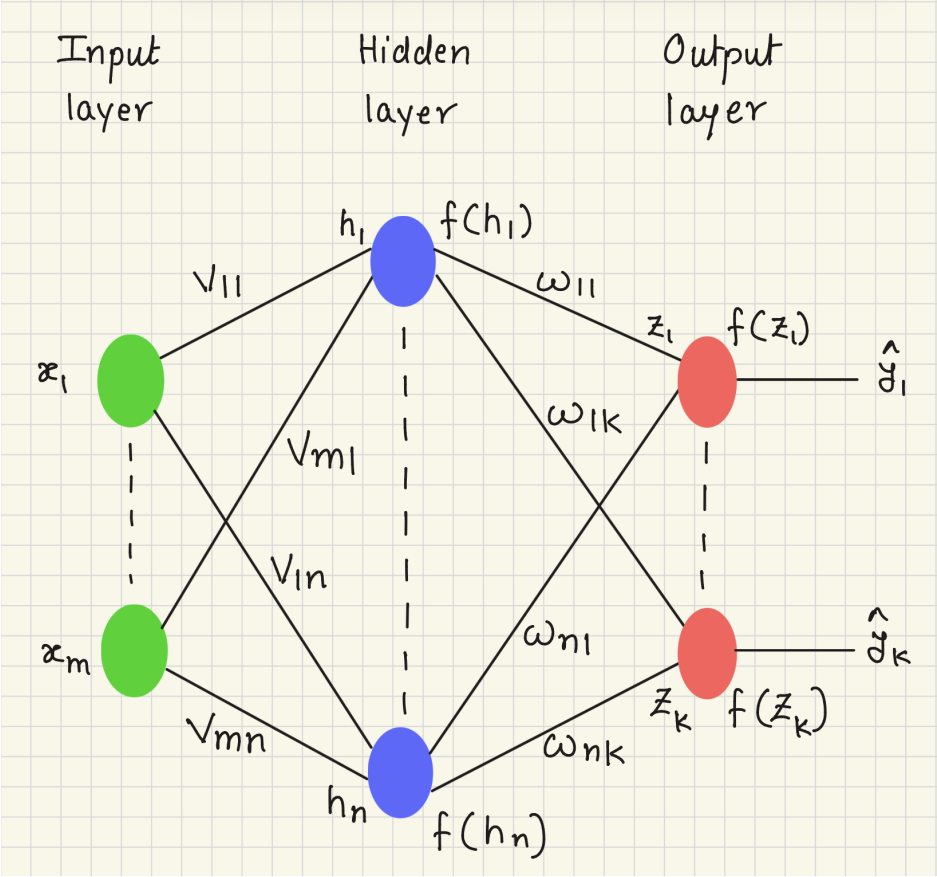

Figure 4: A simple neural network.

2 Gradient of population cross-entropy loss

Consider the neural network given in figure 4, with ReLU activation functions (denoted by f) on all neurons and softmax activation function in the output layer.

Given an input $[x_1, x_2, ..., x_m]^T$, the hidden units in the network are activated in stages as described by the following system of equations:

$$\begin{pmatrix} v_{11} & v_{21} & \dots & v_{m1} \\ v_{12} & v_{22} & \dots & v_{m2} \\ \vdots & \vdots & \vdots & \vdots \\ v_{1n} & v_{2n} & \dots & v_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{pmatrix} + \begin{pmatrix} v_{01} \\ v_{02} \\ \vdots \\ v_{0n} \end{pmatrix} = \begin{pmatrix} h_1 \\ h_2 \\ \vdots \\ h_n \end{pmatrix}$$

$$\begin{array}{rcl} f(h_1) & = & max\{h_1, 0\} \\ f(h_2) & = & max\{h_2, 0\} \\ \vdots & & \vdots \\ f(h_n) & = & max\{h_n, 0\} \end{array}$$

$$h_i = v_{0i} + \sum_{j=1}^{m} v_{ji} x_j, \qquad 1 \le i \le n \tag{37}$$

$$\begin{pmatrix} w_{11} & w_{21} & \dots & w_{n1} \\ w_{12} & w_{22} & \dots & w_{n2} \\ \vdots & \vdots & \vdots & \vdots \\ w_{1k} & w_{2k} & \dots & w_{nk} \end{pmatrix} \begin{pmatrix} f(h_1) \\ f(h_2) \\ \vdots \\ f(h_n) \end{pmatrix} + \begin{pmatrix} w_{01} \\ w_{02} \\ \vdots \\ w_{0k} \end{pmatrix} = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_k \end{pmatrix}$$

$$\begin{array}{rcl} f(z_1) & = & max\{z_1, 0\} \\ f(z_2) & = & max\{z_2, 0\} \\ \vdots & & \vdots \\ f(z_k) & = & max\{z_k, 0\} \end{array}$$

$$z_i = w_{0i} + \sum_{j=1}^{n} w_{ji}\, f(h_j), \qquad 1 \le i \le k. \tag{38}$$

The final output of the network is obtained by applying the softmax function to the last hidden layer,

$$\hat{y} = \begin{pmatrix} \hat{y_1} \\ \hat{y_2} \\ \vdots \\ \hat{y_k} \end{pmatrix} \quad where \quad \hat{y_i} = \frac{e^{f(z_i)}}{\sum_{j=1}^{k} e^{f(z_j)}} \qquad 1 \le i \le k.$$

Assume that V is a constant matrix and also, for simplicity, assume that $f(z_i) = z_i$ and $f(h_i) = h_i$ to stay focused on computing gradient of cross-entropy without bringing in ReLU.

Then cross-entropy loss for a single example is

$$L(\theta) = L(W) = -\sum_{i=1}^{k} y_i\, log(\hat{y_i}). \tag{39}$$

and the gradient of L with reference to W is

$$\nabla_W L = \begin{pmatrix} \frac{\partial L}{w_{11}} & \frac{\partial L}{w_{21}} & \cdots & \frac{\partial L}{w_{n1}} \\ \frac{\partial L}{w_{12}} & \frac{\partial L}{w_{22}} & \cdots & \frac{\partial L}{w_{n2}} \\ \vdots & \vdots & \vdots & \vdots \\ \frac{\partial L}{w_{1k}} & \frac{\partial L}{w_{2k}} & \cdots & \frac{\partial L}{w_{nk}} \end{pmatrix}$$

Now consider

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial}{\partial w_{21}} \left[-\sum_{i=1}^{k} y_i \, log(\hat{y}_i) \right] \tag{40}$$

$$= \sum_{i=1}^{k} -y_i \frac{\partial \, log(\hat{y}_i)}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_{21}} \tag{41}$$

$$= \sum_{i=1}^{k} -y_i \frac{1}{\hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_1} \frac{\partial z_1}{\partial w_{21}} \tag{42}$$

Now,

$$\frac{\partial z_1}{\partial w_{21}} = \frac{\partial}{\partial w_{21}} \left(w_{01} + \sum_{j=1}^{n} w_{j1} h_j \right) \tag{43}$$

$$= h_2 \tag{44}$$

and we have two cases for $\frac{\partial \hat{y}_i}{\partial z_1}$.

Case 1: $i = 1$

$$\frac{\partial \hat{y}_1}{\partial z_1} = \frac{\partial}{\partial z_1} \left(\frac{e^{z_1}}{\sum_{j=1}^{k} e^{z_j}} \right) \tag{45}$$

$$= \frac{e^{z_1} \sum_{j=1}^{k} (e^{z_j}) - e^{z_1} \, e^{z_1}}{(\sum_{j=1}^{k} (e^{z_j}))^2} \tag{46}$$

$$= \left(\frac{e^{z_1}}{\sum_{j=1}^{k} (e^{z_j})} \right) - \left(\frac{e^{z_1}}{\sum_{j=1}^{k} (e^{z_j})} \right)^2 \tag{47}$$

$$= \hat{y}_1 - (\hat{y}_1)^2 \tag{48}$$

Case 2: $i \neq 1$

$$\frac{\partial \hat{y}_i}{\partial z_1} = \frac{\partial}{\partial z_1}\left(\frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}\right) \tag{49}$$

$$= \frac{0 - e^{z_i}\, e^{z_1}}{(\sum_{j=1}^{k}(e^{z_j}))^2} \tag{50}$$

$$= -\hat{y}_i\, \hat{y}_1 \tag{51}$$

Now plugging back into the equation for $\frac{\partial L}{\partial w_{21}}$

$$\frac{\partial L}{\partial w_{21}} = -(y_1(1 - \hat{y_1}) - \hat{y_1}\, y_2 - ... - \hat{y_1}\, y_k)\, h_2 \tag{52}$$

$$= -\left(y_1 - \hat{y_1}\, y_1\ - \hat{y_1}\, y_2 - ... - \hat{y_1}\, y_k\right)\ h_2 \tag{53}$$

$$= -(y_1 - \hat{y_1}(y_1 + y_2 + ... + y_k))\, h_2 \tag{54}$$

$$= (\hat{y_1} - y_1)\, h_2 \tag{55}$$

since $y_1 + ... + y_k = 1$.

It is easy to see that this generalizes to

$$\frac{\partial L}{\partial w_{ji}} = (\hat{y_i} - y_i)\, h_j \tag{56}$$

which is nothing but the difference between the predicted and the ground truth probability distributions, scaled by the embeddings. This simplicity makes population cross-entropy along with softmax a popular combination for transformer models.